Voyage-AI — Engineering Methodology & Operational Workflows

Document Reference: `docs/reports/methodology.pdf` / `docs/reports/methodology.md`
System Version: 0.1.0 (Phase 1 Baseline)
Author: Senior Systems Analyst & QA Lead
Verification Date: October 21, 2026
Status: Implementation-Verified

---

1. Development Methodology & Engineering Lifecycle

1.1 Evidenced Engineering Process

The Voyage-AI engineering lifecycle follows a disciplined, evidence-based phased development model governed by extensive pre-implementation architecture audits (`docs/audit/PRE_IMPLEMENTATION_AUDIT_v1.1.md`) and a formal phased roadmap (`docs/IMPLEMENTATION_PLAN.md`).

The development methodology is characterized by three core tenets:
1. **Vertical Slice Verification Early:** Rather than authoring speculative backend layers or disconnected UI mockups, the engineering process requires building functional, testable vertical slices at each milestone.
2. **Offline-First Deterministic Testing:** Development and automated verification do not depend on uncontrolled external network services or live third-party APIs. External network boundaries are strictly encapsulated behind provider adapters with mock server fixtures and simulated network latency.
3. **Database Isolation & Safety Guarantees:** Test harnesses are programmatically prevented from contacting production or developer databases. Automated integration tests enforce execution against dedicated in-memory database instances (`jdbc:h2:mem:voyage_flyway_test`).

1.2 Module Implementation Sequence

The Git commit history (`git log`) and repository structure confirm the following sequential implementation progression:

```
[Phase 0: Foundation Setup]
  |  ├── Repository scaffolding (`frontend/`, `backend/`, `infra/`, `tests/`)
  |  ├── Spring Boot 4.1.1 baseline & embedded Tomcat configuration (Port 8081)
  |  ├── Flyway migration baseline (`V0__baseline.sql`) & H2 test harness
  |  └── Custom health controller (`/api/v1/health`) and Spring Actuator (`/actuator/health`)
  v
[Phase 1: Design System & Discovery Prototype]
  |  ├── Emil Kowalski & Impeccable design token definition in Tailwind CSS
  |  ├── Tactile component library (`Button.tsx`, `Badge.tsx`, `SkeletonCard.tsx`, `EmptyState.tsx`)
  |  ├── Responsive navigation and live health indicator (`Navbar.tsx`)
  |  ├── Curated Indian destination dataset (`mockDestinations.ts`: Jaipur, Varanasi, Munnar, Hampi)
  |  └── Multi-criteria discovery filtering & 5-tab detail modal dialog (`DestinationDetailModal.tsx`)
  v
[Integration Slice: Meteorological Provider]
  |  ├── Domain adapter interface (`WeatherProvider.java`)
  |  ├── Open-Meteo REST client integration with bounded HTTP timeouts (`OpenMeteoWeatherService.java`)
  |  ├── Controller exposure (`WeatherController.java` → `/api/v1/weather`)
  |  └── Comprehensive unit test suites (7 controller tests, 6 service tests, 7 frontend tests)
  v
[Phase 2 Target: Relational Persistence & Security] (Current Next Step)
     ├── Flyway relational schema migration (`V1__init_schema.sql`)
     ├── Spring Data JPA entity repositories (`users`, `destinations`, `trips`)
     └── Stateless JWT authentication filter chain (`JWT_AUTH_DESIGN.md`)
```

1.3 Version Control & Branching Strategy

- **Branch Topology:** Development is conducted directly on the `main` trunk, maintaining an always-buildable, test-passing baseline.
- **Commit Conventions:** Follows semantic commit prefixes:
- `feat:` New user-facing capabilities or service integrations.
- `chore:` Configuration, dependencies, and environment hygiene.
- `docs:` Architectural and requirement specifications.
- **Working Tree Cleanliness:** Verified via `git status` with zero uncommitted artifacts or transient build pollution.

---

2. End-to-End Operational Workflows

2.1 Workflow 1: Destination Discovery & Multi-Criteria Filtering



Step-by-Step Execution:

1. **Initiating Request:** User navigates to `http://localhost:5173` or clicks a filter button (Region, Travel Style, or Budget Tier) or types into the search bar.
2. **Input Validation:** Input text is sanitized and normalized to lowercase; empty filter tags default to `'All'`.
3. **Processing Logic:** `App.tsx` executes a memoized filter predicate (`useMemo`) over `MOCK_DESTINATIONS`. A brief 120ms artificial debounce displays tactile shimmer skeletons (`SkeletonCard.tsx`) to provide visual continuity.
4. **Card Rendering:** Matching destinations render as `DestinationCard` components displaying high-resolution imagery, temperature chips, climate tags, stay/dining counts, and transparent daily budget estimates.
5. **Detail Inspection:** Clicking a destination card opens `DestinationDetailModal`, rendering 5 organized tabs:
6. *Overview:* Tagline, state, recommended duration, best visiting months, and value highlights.
7. *Curated Stays:* Royal havelis, plantation villas, and boutique retreats with illustrative per-night rates.
8. *Dining Trails:* Regional culinary specialties, signature thalis, must-try items, and honest price tiers.
9. *Attractions:* Heritage monuments, architectural landmarks, and cultural sites.
10. *Honest Budget Breakdown:* Daily cost estimates across Budget, Comfort, and Luxury tiers with transparent two-traveler totals.

11. **Recovery & Empty States:** If filters match zero destinations, `EmptyState.tsx` displays an encouraging recovery card with a one-click "Reset all filters" button.

2.2 Workflow 2: Weather Forecast Retrieval & Provenance Tracking

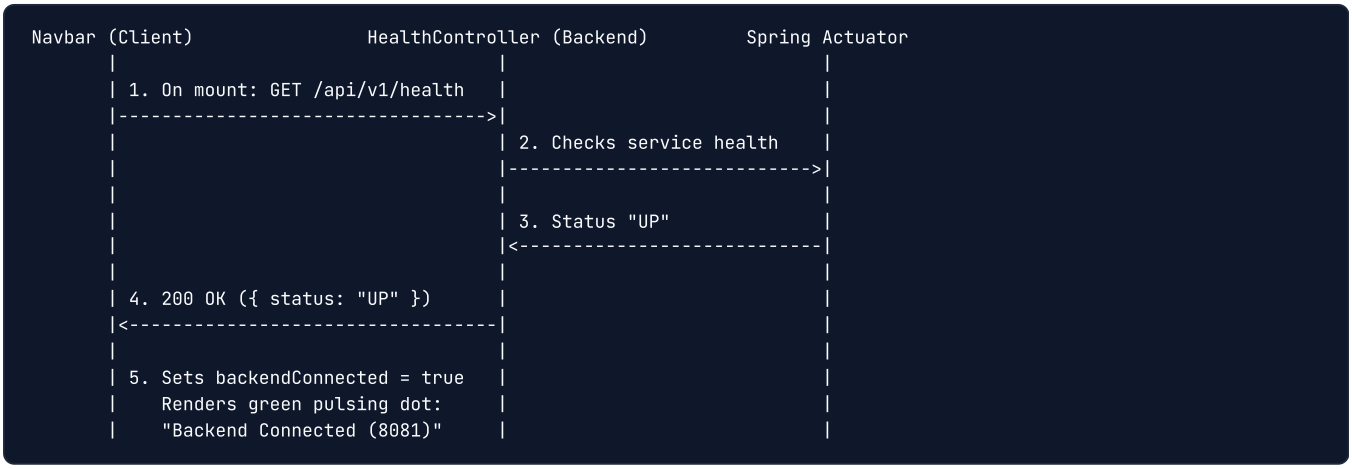


Step-by-Step Execution:

1. **Trigger:** Client dispatches an HTTP `GET` request to `/api/v1/weather` with query parameters `latitude` and `longitude`.
2. **Controller Validation:** `WeatherController` verifies parameter presence; missing parameters immediately trigger a `400 Bad Request` with error code `MISSING_PARAMETER` via `GlobalExceptionHandler`.
3. **Service Validation:** `OpenMeteoWeatherService` validates that coordinates fall within valid terrestrial bounds (`[-90.0, 90.0]` for latitude, `[-180.0, 180.0]` for longitude). Violations throw `IllegalArgumentException`, mapped to `400 Bad Request` (`INVALID_ARGUMENT`).
4. **HTTP Transport:** The service constructs an outbound HTTP request using Spring's `RestClient` with explicit connect (3000ms) and read (5000ms) timeouts configured via `SimpleClientHttpRequestFactory`.
5. **Provenance Annotation:** The response is mapped into `WeatherResponse` containing:
6. `CurrentWeatherDTO` : Temperature, apparent temperature, relative humidity, wind speed, precipitation, WMO weather code, and human-readable weather condition.
7. `DailyForecastDTO` : Date-indexed daily high/low temperatures.
8. `WeatherUnitsDTO` : Metric units (`°C` , `km/h` , `mm` , `%`).
9. `WeatherProvenanceDTO` : `sourceProvider: "Open-Meteo"` , retrieval timestamp, license (`"CC-BY 4.0"`), and attribution (`"Weather data by Open-Meteo.com"`).
10. **Error & Degradation Handling:**
11. Network timeout or unreachable host throws `WeatherServiceException` with code `WEATHER_PROVIDER_TIMEOUT` (HTTP 503).
12. Upstream 5xx or 429 rate limit throws `WEATHER_PROVIDER_ERROR` (HTTP 503).
13. Empty or null response body throws `WEATHER_DATA_UNAVAILABLE` (HTTP 503).

14. Under no circumstances is synthetic fallback weather generated.

2.3 Workflow 3: Live Backend Health Monitoring

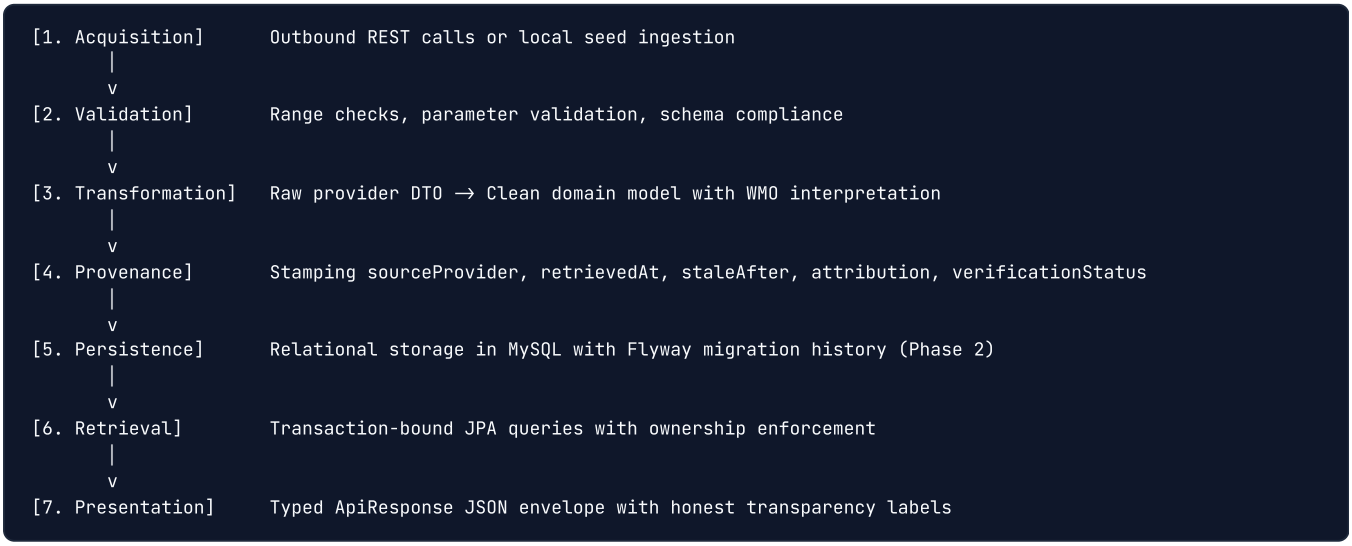


1. **Trigger:** When `App.tsx` mounts, a `useEffect` hook triggers a non-blocking `fetch()` request to `${apiBaseUrl}/health`.
2. **Execution:** `HealthController` assembles service metadata (application name, active profile, version).
3. **Client UI State:** If the response is `200 OK`, `backendConnected` state is set to `true`, rendering a subtle green indicator dot in the `Navbar`. If the request fails or times out, the indicator displays a neutral amber state indicating standalone client operation.

---

3. Data-Processing Methodology

Voyage-AI enforces a strict 7-stage data processing pipeline designed around authenticity and provenance:



---

4. External Service Integration Methodology

4.1 Integration Characteristics & Failure Policies

Integration	Primary Responsibility	Request Construction	Timeout & Quota Policy	Fallback Behavior
Open-Meteo	Live meteorological conditions and forecasts	REST `GET` with coordinate parameters; accepts JSON	Connect: 3,000ms; Read: 5,000ms; 10,000 requests/day	Returns HTTP 503 with explicit error envelope; never fabricates dummy weather
OpenStreetMap	Interactive vector tile rendering	Standard Leaflet tile layer requests (`https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png`)	Respects HTTP cache headers; bulk downloading prohibited	Client gracefully degrades to static map cards
Wikimedia Commons	High-resolution cultural photography	REST API lookup for public domain / Creative Commons media	User-Agent required; fair use rate limiting	Falls back to styled SVG monument placeholder with attribution notice
OSRM	Routing distances and transit travel times	REST query with coordinate pairs	Public demo server without commercial SLA	Falls back to spherical Haversine direct distance calculation
Google Gemini 1.5	Itinerary proposal sequencing (Grounded RAG)	Entity-constrained JSON schema prompt with retrieved DB IDs	15 RPM / 1,500 RPD free tier ceiling; strictly 0 PII	Falls back to unassisted manual itinerary sequencing

---

5. Testing & Verification Methodology

5.1 Multi-Tiered Testing Pyramid

The automated verification strategy is organized into five complementary tiers:

```
            / \
           /   \          Tier 5: Centralized Java Playwright Browser Tests
          / E2E \         (Desktop & Mobile Viewport, Navigation, Filter State)
         /-------\
        /   API   \       Tier 4: Black-Box REST API Tests
       /  Testing  \      (Direct HTTP checks on /api/v1/health & /weather)
      /-------------\
     /   Database    \    Tier 3: Flyway & Persistence Integration Tests
    /   Integration   \   (Isolated H2 MySQL mode, Zero Host Mutation)
   /-------------------\
  /   Frontend Unit     \ Tier 2: Vitest + React Testing Library
 /     (Component DOM)   \ (Tactile Buttons, Skeletons, Modals, Fallbacks)
/-------------------------\
/       Backend Unit        \ Tier 1: JUnit 5 + MockMvc + Mockito
/   (Controller & Service)   \ (HTTP mapping, Error codes, Coordinate validation)
-----------------------------
```

5.2 Verification Commands & Environment Prerequisites

```
# 1. Run Backend Unit, Controller & Database Integration Tests (20 Tests):
cd backend
.\mvnw.cmd test

# 2. Run Frontend Component & Unit Tests (7 Tests):
cd frontend
npm run test:unit

# 3. Run Centralized Java Playwright Browser & Black-Box API Tests:
# Prerequisites: Backend running on :8081, Frontend running on :5173
cd tests
mvn test

# 4. Run Unified Test Automation Runner (Executes all suites sequentially):
.\scripts\run-all-tests.ps1
```

---

6. Engineering Principles & Rationale

1. **Honest Data Provenance:** Software systems often fabricate placeholder data to create an illusion of completeness. Voyage-AI rejects this practice: if an attraction does not have an official entry fee, it is displayed as `null` with a clear label, building genuine user trust.
2. **Tactile Interaction & Micro-Feedback (Emil Kowalski Philosophy):** Interactive elements must provide immediate physical feedback. Buttons compress (`:active:scale(0.97)`) with restrained 160ms transitions (`cubic-bezier(0.23, 1, 0.32, 1)`). Modals expand outward from triggers with ESC key dismissability and automated focus restoration.
3. **WCAG 2.2 AA Accessibility Compliance:** All color combinations meet or exceed 4.5:1 contrast ratios. Interactive elements maintain visible 2px focus rings (`focus-visible:ring-2 focus-visible:ring-brand-600`), ensuring full screen-reader and keyboard navigation.